Package onboarding and service seams

SPEC-ARCH-14 v0.1 OVERVIEW Jeremie Nunez

1 Why this document exists

The monorepo now has five first-class packages with different ownership rules: `@interview/thalamus`, `@interview/sweep`, `@interview/shared`, `@interview/db-schema`, and `@interview/cli`. The goal of this note is twofold. First, it turns package READMEs into a stable onboarding contract: every package must say what it owns, what it explicitly does not own, and how it is extended without violating the architecture. Second, it makes the service seams explicit. If a package can later become its own process, it must already depend on ports and contracts rather than hidden in-process shortcuts.

The package-onboarding writing standard now lives in `docs/refactor/package-onboarding-standard.md`. The five package READMEs use the same structure: abstract, responsibilities, non-responsibilities, architecture, public API, extension points, runtime dependencies, anti-patterns, tests, and change rules. The intent is not marketing. The intent is to make architectural ownership reviewable.

2 Workspace dependency graph

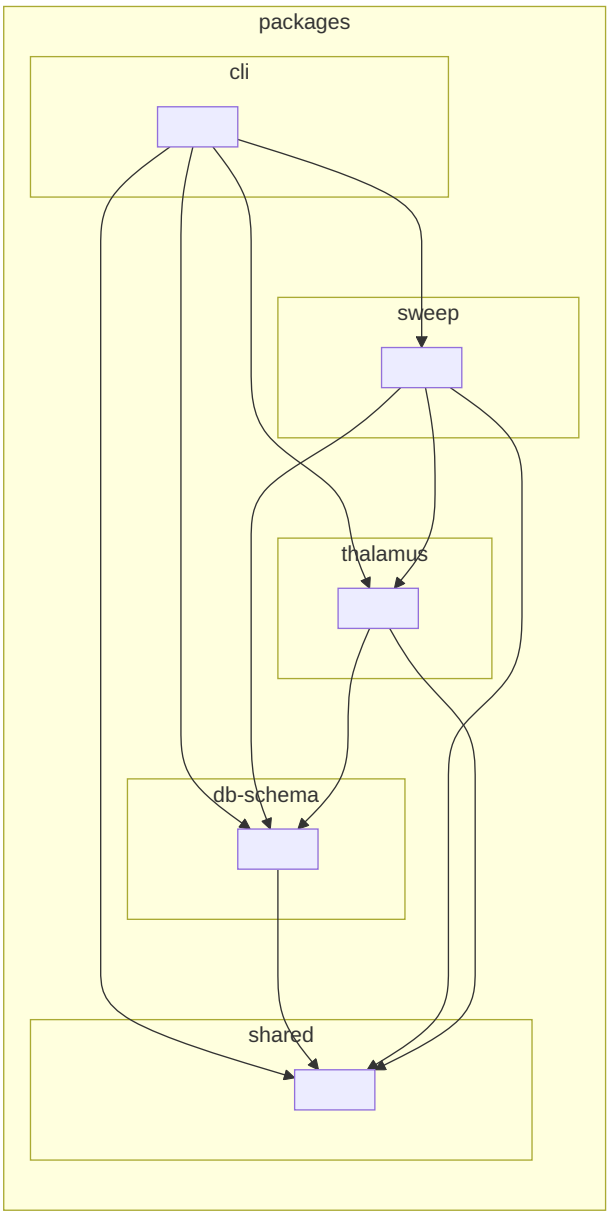

Figure 1: Package-level dependency graph generated from dependency-cruiser. The graph is intentionally acyclic at the package boundary: `shared` and `db-schema` sit at the bottom, `thalamus` and `sweep` build on them, and `cli` composes the runtime as a client.

This graph is the shortest proof that the package split is real. `shared` and `db-schema` are foundational. `thalamus` depends on both to run the research kernel. `sweep` depends on both and also on `thalamus` because the current review kernel still reuses cortex-facing building blocks. `cli` depends on all of them because it is a client and composition root, not a kernel. The service-readiness implication is direct: `thalamus` and `sweep` can become separate services only if they continue to consume each other through explicit contracts or HTTP boundaries, not through ad-hoc imports

or direct database shortcuts.

3 Package writing contract

Every package README now answers three questions.

1. What does the package do?
2. What does it own, and what does it not own?
3. How is it extended without breaking the boundary?

The standard is deliberately strict. Each package README must expose the same sections and must use stable vocabulary: *owns vs. does not own*, *public API vs. internal detail*, *generic vs. domain-specific*, and *port vs. adapter*. This matters because onboarding docs become part of the architectural review surface. If a change cannot be described cleanly in those terms, it is often a sign that the package boundary is leaking.

4 @interview/thalamus

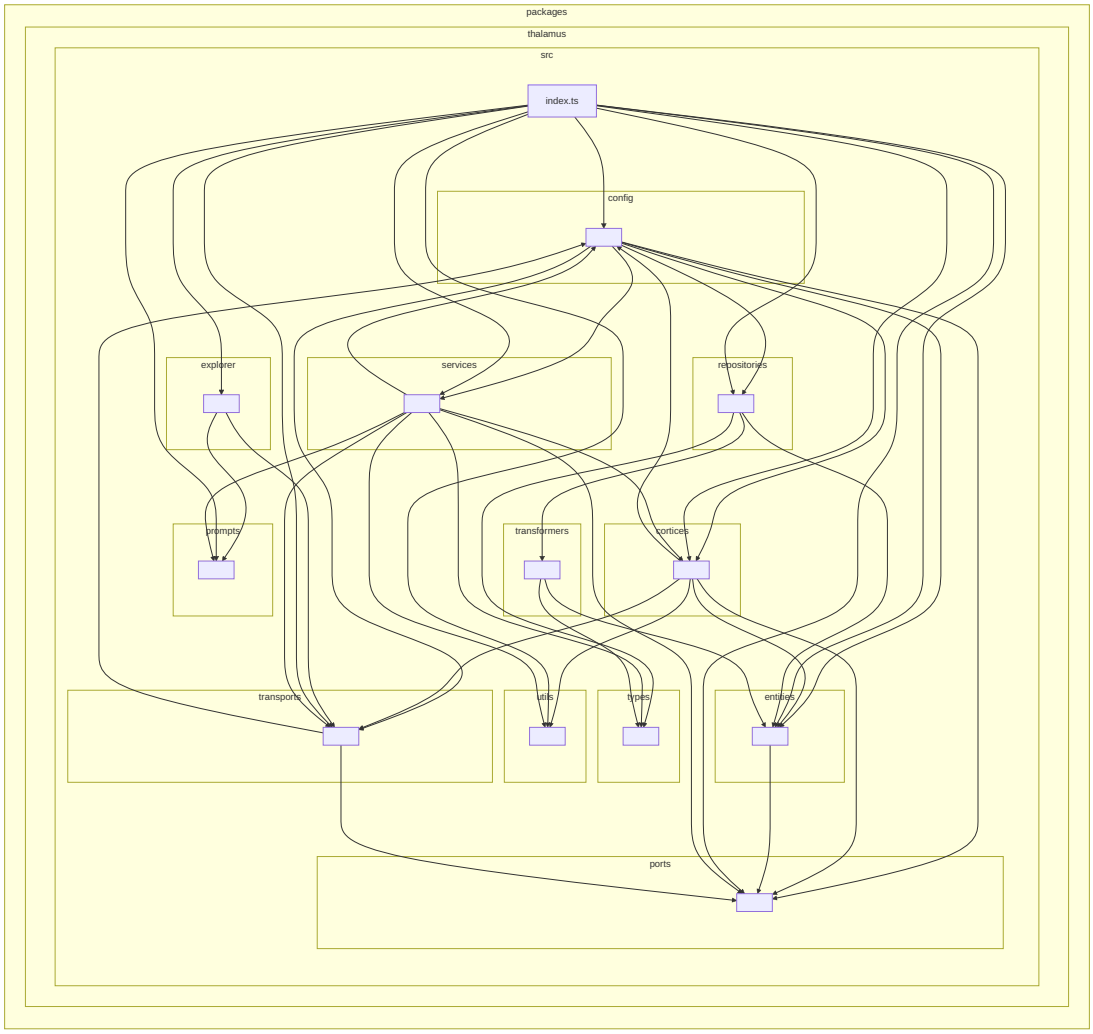

Figure 2: Dependency-cruiser view of `thalamus`. The package fans from configuration and ports into cortices, services, repositories, transports, and utilities. The diagram shows a generic kernel with explicit seams rather than an app-owned service bag.

`thalamus` is the research-cycle kernel. It owns planner/executor orchestration, cortex dispatch, graph persistence, reflexion, stop criteria, and generic verification signals. It does *not* own SSA policy, app-owned prompts, UI DTOs, or direct access to application tables beyond the `research_*` graph boundary. Those behaviors enter through `DomainConfig`, `CortexDataProvider`, `EntityCatalogPort`, `SourceFetcherPort`, and `WebSearchPort`.

The key architectural move is that the package can run the cycle without knowing the downstream domain. `buildThalamusContainer` wires repositories, strategies, registry, planner, executor, reflexion, and graph service. The app provides the domain profile and the skill pack. This is the difference between an LLM workflow and a reusable kernel: domain behavior is injected, not hidden in the package. In service terms, `thalamus` is the strongest candidate for an isolated service because its stable contract is already the run-cycle and graph boundary rather than an app controller.

Public surface that matters: `buildThalamusContainer`, `ThalamusService`, `registerThalamusConfigDomai` the port types, and the transport / nano-swarm helpers. The README explicitly forbids hard-coding SSA cortex names, reading app tables directly, or requiring deep imports for new public symbols.

5 @interview/sweep

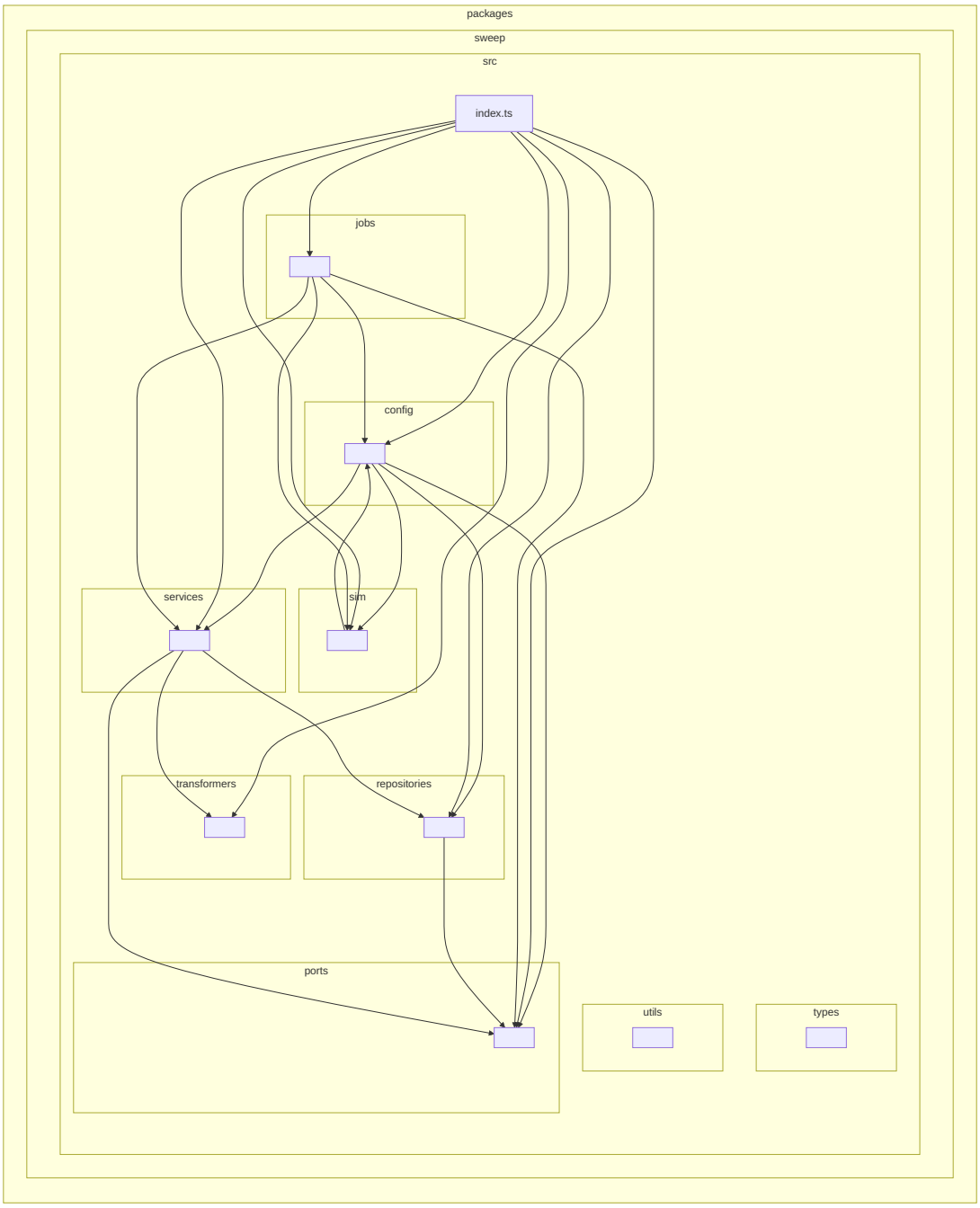

Figure 3: Dependency-cruiser view of `sweep`. The center of gravity is the review kernel plus the simulation subgraph. Ports and the container isolate app-owned promotion, audit, and resolution behavior.

`sweep` is the audit and review kernel. It owns the nano-sweep loop, review-state persistence in Redis, the resolution service, worker factories, queues, and the simulation engine used for verification and counterfactual exploration. It does *not* own app-specific finding schemas, promotion logic, or application database writes. The app injects those through `FindingDomainSchema`, `DomainAuditProvider`, `SweepPromotionAdapter`, `ResolutionHandlerRegistry`, and the sim ports.

This package matters for service decomposition because it already behaves like a specialized worker

service. The review loop and the sim engine are both process oriented, queue backed, and adapter driven. The service-readiness question is therefore not whether it can move out of process, but whether all remaining cross-package shortcuts have been reduced to ports or HTTP adapters. The package-onboarding doc makes that constraint explicit: if a feature needs app writes or domain naming, the adapter stays in the app and the package surface stays generic.

Public surface that matters: `buildSweepContainer`, `NanoSweepService`, `SweepResolutionService`, `SweepRepository`, the worker factories, and the sim kernel exports. The README forbids putting app-owned database code into sweep services, hardcoding a domain schema, or bypassing ports from workers.

6 `@interview/shared`

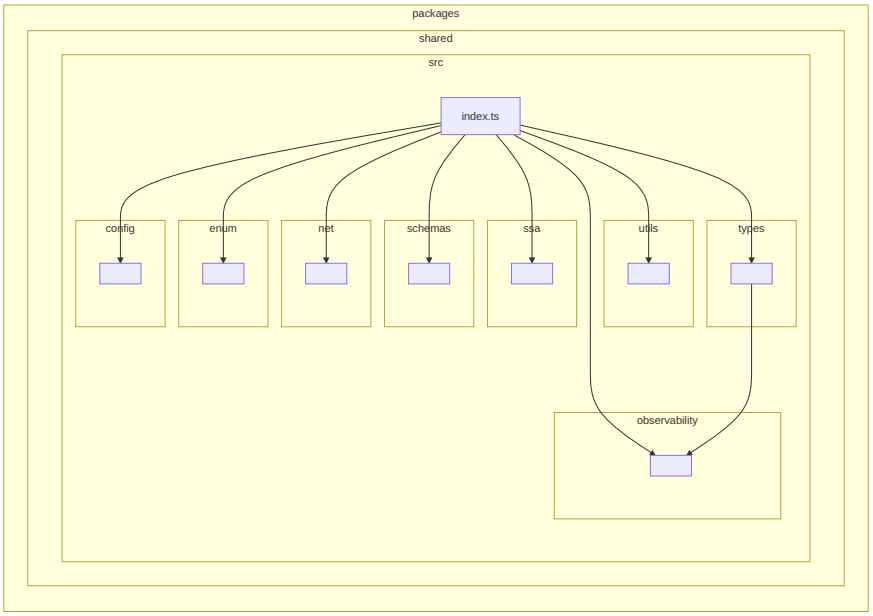

Figure 4: Dependency-cruiser view of `shared`. It is intentionally small: a root barrel over enums, config, observability, schemas, SSA-facing DTO views, and generic utilities.

`shared` is the contract layer. It owns stable enums, shared types, stream contracts, observability primitives, and generic helpers such as structured errors, concurrency limiting, and safe fetch wrappers. It does *not* own business workflows, storage policy, package composition, or package-local

orchestration. The rule is simple: a symbol belongs here only if multiple packages rely on the same stable contract.

That smallness is strategic. `shared` is at the bottom of the dependency graph, so any leak here propagates everywhere. This is why the README says not to move policy here just because two files look similar. Shared code must remain low-level and infrastructurally neutral. If the package grows into a dumping ground, the entire service decomposition story becomes less credible because the real architecture would be encoded in an unowned utility bag.

Public surface that matters: `ReplStreamEvent` and follow-up plan types, shared enums, step telemetry, `AppError`, `tryAsync`, `mapWithConcurrency`, runtime-config contracts, and shared Zod-backed view models. The README requires additive, cross-repo-safe changes because the blast radius of breaking a shared type is high.

7 @interview/db-schema

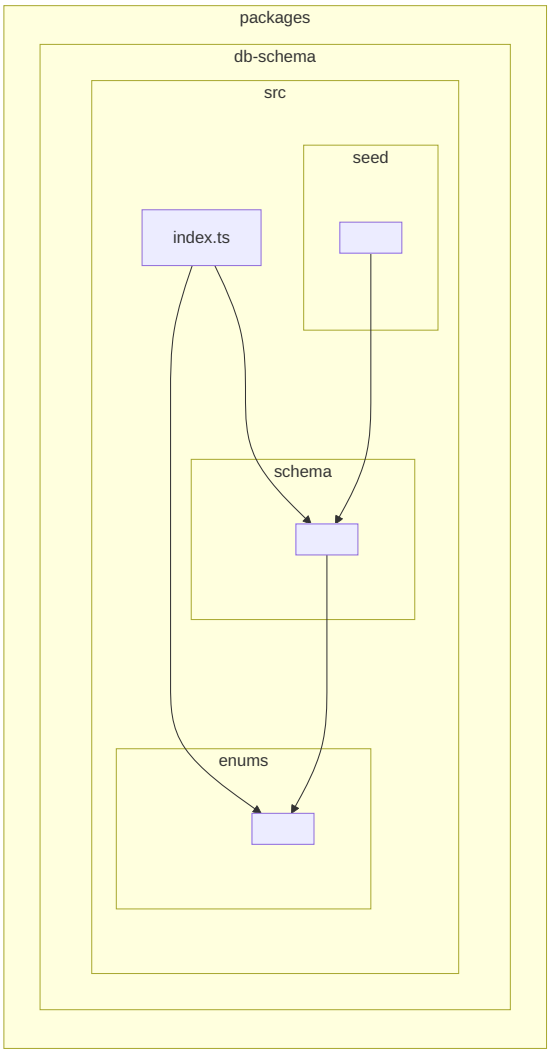

Figure 5: Dependency-cruiser view of `db-schema`. The package is intentionally simple: enums feed schema, schema feeds seeds, and the root barrel exports the storage contract.

`db-schema` is the storage-contract package. It owns the Drizzle schema, Postgres enums, inferred row and insert types, migrations, and seed or maintenance scripts. It does *not* own repositories,

services, LLM workflows, or application policy. This package answers one question only: what is the persistent shape of the system?

That makes it the second foundational service seam after `shared`. If a future split places the database-facing contract behind its own migration and release process, this package already defines the boundary. The two main rules are therefore strict. First, storage-shape changes land with migrations in the same diff. Second, raw SQL migrations belong here when Drizzle code generation is insufficient. The onboarding doc makes both rules explicit so operational knowledge does not stay tribal.

Public surface that matters: `Database`, the schema barrels, the exported tables, the enum barrels, and the inferred types. The README forbids putting repository logic here or recreating enum or row shapes in another package.

8 @interview/cli

Figure 6: Dependency-cruiser view of `cli`. The package is a client shell: entrypoint, boot wiring, router, renderers, and terminal components.

`cli` is not a kernel. It is the terminal client and composition root for the operator REPL. It owns the Ink application, command routing, adapter composition, fixture-aware local behavior, and terminal UX concerns such as logs and ETA tracking. It does *not* own business policy, storage contracts, or backend kernels. Any remaining direct Drizzle reads in `buildRealAdapters` are migration debt, not a target design.

This distinction matters because clients often accrete business logic during a demo phase. The package-onboarding doc prevents that drift by making adapter ownership explicit. New domain behavior should arrive through adapter methods or HTTP endpoints, not through direct imports of app-owned SSA code into the UI layer. In a microservice future, `cli` stays a client and simply points at networked backends.

Public surface that matters: `main`, `buildRealAdapters`, the adapter interface, and the package scripts. The README explicitly rejects new direct database reads for domain features.

9 Microservice implications

The package split is now strong enough to support a realistic service roadmap. `thalamus` can become a research orchestration service. `sweep` can become a review and simulation service. `console-api` can stay the app-owned façade that injects SSA policy and coordinates HTTP contracts. `shared` and `db-schema` remain libraries, not services.

What still matters is discipline. A package boundary only predicts a clean service boundary if three conditions remain true:

1. no hidden fallback policy in the generic packages,
2. no direct reads of app-owned tables from the kernels,
3. no public behavior reachable only through deep imports.

The new package README standard is therefore not documentation polish. It is a maintenance mechanism for the service seams. If a future diff changes ownership, it should have to update the package README as well. That keeps the architecture reviewable in plain text before it becomes expensive in runtime topology.